

Année universitaire	2025-2026		
Département	Informatique	Année	M1 DataEng
Matière	Concepts, fonctionnements et technologies de l'IA		
Enseignant	Haytham Elghazel		
Intitulé TD/TP :	Atelier 4 : Apprentissage supervisé avec Python		
Contenu	• Data Preprocessing (données hétérogènes, données manquantes, etc.) • Feature engineering • Feature selection • Classification • Evaluation de la qualité d'un classifieur		

Dans cet atelier pratique, vous allez expérimenter des algorithmes de traitement de données pour répondre à différents problèmes liés à l'apprentissage supervisé avec le langage **Python**.

Créer un nouveau notebook Python et taper le code suivant dans une nouvelle cellule :

```
import numpy as np
np.set_printoptions(threshold=10000,suppress=True)
import pandas as pd
import warnings
import matplotlib.pyplot as plt
warnings.filterwarnings('ignore')
```

Apprentissage supervisé : Feature engineering et Classification

L'objectif dans cette partie est de construire un bon classifieur sur un jeu de données de *credit scoring* du fichier **"credit_scoring.csv"**.

1. **Chargement des données et préparation** : Dans un premier temps nous allons importer le jeu de données et analyser ses caractéristiques.
 - Importer ce jeu de données avec la librairie **pandas** (c.f. **read_csv**)
 - Transformer votre jeu de données issue de **pandas** qui sera de type **Data Frame** en **numpy Array** (c.f. **values**) et séparer ensuite les variables caractéristiques de la variable à prédire (**status**) en deux tableaux différents.
 - Analyser les propriétés de vos données : taille de l'échantillon (c.f. **shape**), pourcentage d'exemples positifs et négatifs.
 - Pour éviter d'avoir un résultat biaisé du classifieur que nous allons construire, séparer les données en deux parties (de taille **50%** chacune) une dite d'apprentissage qui servira à l'apprentissage du classifieur et l'autre dite de test qui servira à son évaluation (c.f. **train_test_split** avec un **random_state=1**).
2. **Fixer une variable SEED = 1**
3. **Apprentissage et évaluation de modèles** : Utiliser ensuite sur votre jeu de données les algorithmes d'apprentissage supervisé suivants :
 - Un arbre CART avec **max_depth=3**
 - Un arbre ID3 avec **max_depth=3**
 - MultilayerPerceptron à deux couches de tailles respectives 20 et 10 par exemple
 - k-plus-proches-voisins avec **k=5** par exemple
 - Bagging avec 200 arbres de décision par exemple
 - AdaBoost avec 200 arbres de décision par exemple
 - Random Forest avec 200 arbres de décision par exemple

- Etc.

Pour tous les algorithmes qui le permettent, fixez `random_state = SEED` afin d'assurer la reproductibilité des résultats, et ajoutez `n_jobs = -1` pour activer la parallélisation et utiliser tous les cœurs du processeur. Attention, de dernier paramètre `n_jobs` doit être fixé à 1 dans le cas d'une mise en production pour assurer la stabilité de la solution.

Remarque : Si vous ne connaissez pas le fonctionnement précis de l'un de ces algorithmes, n'hésitez pas à consulter la documentation de Scikit-learn. Attention, la plupart de ces algorithmes ont des paramètres qu'il vous faudra prendre en compte lors de vos expérimentations. Vous en choisirez quelques-uns que vous ferez varier afin d'observer l'effet pratique de ces paramètres sur les résultats obtenues (par exemple `k` pour les *k-plus-proches-voisins* ou le nombre d'arbres dans un Random Forest).

L'objectif est à présent de comparer les résultats obtenus à l'aide de ces algorithmes sur ce jeu de données. Cette comparaison doit être faite dans une **fonction python** et s'appuiera sur :

- la **matrice de confusion**,
- l'estimation de **la moyenne de l'accuracy et le meilleur critère entre le Rappel et la Précision** (rappel ou précision) dont vous pensez qu'il est le plus adéquat pour ce cas métier de *crédit scoring*,

Dans cette partie, vous devez définir :

- Un **dictionnaire `clfs`** contenant les algorithmes supervisés utilisés.
 - Une **fonction d'évaluation (`evaluate_classifier`)** permettant :
 - d'afficher la matrice de confusion et la courbe ROC avec l'AUC,
 - de calculer l'accuracy, le rappel ou la précision et de retourner un score final donné par la moyenne entre l'accuracy et le meilleur des deux critères selon ce cas métier (accuracy + meilleur critère) / 2.
 - Une **fonction d'apprentissage et de test (`run_classifiers_train_test`)** permettant :
 - d'entraîner chaque algorithme de votre dictionnaire `clfs`,
 - d'afficher les résultats d'évaluation utilisant la fonction `evaluate_classifier`
 - de comparer les scores obtenus,
 - et de retourner le meilleur modèle selon le score final (accuracy + meilleur critère) / 2.
4. **Normalisation des variables continues :** Certains algorithmes d'apprentissage supervisé fonctionneront mieux si les données sont normalisées (centrées autour de 0) pour que toutes les variables caractéristiques aient le même poids dans la phase d'apprentissage. Utiliser le module **`StandardScaler`** de Scikit-learn pour normaliser vos données (il est possible d'utiliser à la place le module **`MinMaxScaler`**). Exécuter à nouveau votre fonction sur vos données une fois normalisées. Interpréter les résultats obtenus en les comparant avec ceux de la question précédente. **Attention à bien utiliser la bonne méthodologie pour normaliser la base de train et la base de test.**
 5. **Création de nouvelles variables caractéristiques par combinaisons linéaires des variables initiales :** Il est parfois utile pour certains classificateurs de faire une réduction de dimensions sur les données afin de déceler et créer certaines combinaisons linéaires dans les variables descriptives et augmenter ainsi le pouvoir discriminant du classifieur. Appliquer une **ACP** (module **`PCA`** de Scikit-learn) sur vos données et garder les **3** premières nouvelles dimensions en les concaténant à vos données normalisées de l'étape précédente. Exécuter à nouveau votre code sur vos nouvelles données. Que se passe-t-il ? **Attention à bien utiliser la bonne méthodologie pour calculer les variables de l'ACP sur la base de train et la base de test.**
 6. Vous avez testé jusqu'à cette étape quelques algorithmes sur une seule base de test, il est nécessaire de rendre la comparaison plus robuste en utilisant une validation croisée.

L'objectif est à présent de comparer les résultats obtenus à l'aide des différents algorithmes donnés ci-dessus sur votre jeu de données. Ces comparaisons s'appuieront sur l'estimation par 10 fold cross-validation validation (c.f. **`cross_val_score`**) du critère **(accuracy+précision)/2**. Il faut afficher aussi la moyenne et l'écart type.

Note :

Pour comparer plusieurs classifieurs sur la même validation croisée, il est recommandé d'utiliser un dictionnaire Python regroupant les classifieurs, par exemple :

```
clfs = {
    'RF': RandomForestClassifier(n_estimators=200, random_state=1),
    'KNN': KNeighborsClassifier(n_neighbors=10),
    liste à compléter
}
kf = KFold(n_splits=10, shuffle=True, random_state=0)
for i in clfs:
    clf = clfs[i]
    cv_acc = cross_val_score(clf, X, Y, cv=kf)
    print("Accuracy for {0} is: {1:.3f} +/- {2:.3f}".format(i, np.mean(cv_acc), np.std(cv_acc)))
```

Vous créerez ensuite une fonction Python ***run_classifiers_cv*** qui automatisera cette comparaison pour tout dictionnaire de classifieurs ***clfs*** et jeux de données donnés (***X*** et sa target ***Y***).

- Exécuter cette fonction sur le jeu de données original, sur les données normalisées et sur les données incluant les variables de l'ACP.
- Identifier le meilleur algorithme selon vos critères.

A partir de cette étape il faut noter quel est le meilleur algorithme que vous allez garder pour la suite et quelle est la meilleure stratégie de préparation de données. Faut-il ou pas normaliser ?

7. **Paramétrage des classifieurs** : Dans cette partie, vous allez utiliser la méthode **GridSearchCV** de scikit-learn afin de tuner les paramètres de votre meilleur algorithme. Vous allez choisir les meilleurs paramètres qui optimisent au mieux le critère suivant : **(accuracy+précision)/2**. Votre code de choix des meilleurs paramètres doit être inclus dans une **fonction Python**
8. **Sélection de variables** : Même si vous utilisez le meilleur algorithme d'apprentissage avec les meilleurs paramètres, la présence de variables bruitées pourra avoir un impact négatif sur les résultats d'apprentissage. La sélection de variables est un processus très important en apprentissage supervisé. Il consiste à sélectionner le sous-ensemble de variables les plus pertinentes (en enlevant le bruit et la redondance) à partir de la série de variables candidates permettant de mieux expliquer et prédire votre target. Dans la suite, vous allez utiliser la méthode **Random Forest** de Scikit-learn pour déterminer quelles sont les meilleures variables pour prédire si une personne va payer son crédit ou pas. Attention :

Ainsi, il faut afficher un histogramme des importances des variables.

Attention : dans la suite de cette partie, pour *X_train* et *X_test*, il faut utiliser les données complètes issues de votre propre choix effectué avant l'étape 7.

Les éléments affichés en vert doivent être ajustés en fonction de vos résultats précédents et de ce qui est demandé dans le TP.

Il est demandé d'inclure les blocs de code ci-dessous dans trois fonctions Python distinctes, afin de structurer votre travail et de favoriser sa réutilisation dans la suite du TP.

Première fonction : Importance des variables

Cette fonction doit permettre de calculer et d'afficher l'importance relative des variables à l'aide d'un modèle Random Forest :

```
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier(n_estimators=1000, random_state=1)
clf.fit(Xtrain, Ytrain)
importances=clf.feature_importances_
```

```

std = np.std([tree.feature_importances_ for tree in clf.estimators_],axis=0)

sorted_idx = np.argsort(importances)[::-1]

features = nom_cols
print(features[sorted_idx])

padding = np.arange(Xtrain.size/len(Xtrain)) + 0.5
plt.barh(padding, importances[sorted_idx],xerr=std[sorted_idx], align='center')
plt.yticks(padding, features[sorted_idx])
plt.xlabel("Relative Importance")
plt.title("Variable Importance")
plt.show()

```

Deuxième fonction – Sélection du nombre optimal de variables

Cette fonction doit permettre de déterminer combien de variables il est pertinent de conserver, en mesurant l'évolution de la performance du **meilleur algorithme** choisi précédemment.

```

KNN=KNeighborsClassifier(n_neighbors=5) // Ici c'est votre Meilleur algorithme
scores=np.zeros(Xtrain.shape[1])
for f in np.arange(0, Xtrain.shape[1]):
    X1_f = Xtrain[:,sorted_idx[:f+1]]
    X2_f = Xtest[:,sorted_idx[:f+1]]
    KNN.fit(X1_f,Ytrain)
    YKNN=KNN.predict(X2_f)
    scores[f]=np.round(accuracy_score(Ytest,YKNN),3)

plt.plot(scores)
plt.xlabel("Nombre de Variables")
plt.ylabel("Accuracy")
plt.title("Evolution de l'accuracy en fonction des variables")
plt.show()

```

(BONUS) Troisième fonction – Explicabilité avec SHAP

Cette fonction doit intégrer une étape d'**explicabilité** à l'aide de la bibliothèque **SHAP** (*SHapley Additive exPlanations*). Elle doit permettre d'interpréter le modèle en identifiant les variables qui influencent le plus les prédictions, aussi bien de manière globale que locale.

9. **Création d'un pipeline** : Dans cette partie vous allez automatiser l'enchaînement des traitements effectués précédemment (Normalisation ou non, ACP ou non et Construction du classifieur et sélection de variables) dans un pipeline (module ***pipeline.Pipeline*** de Scikit-learn). Sauvegarder ensuite votre pipeline dans un **pickle** afin de pouvoir l'utiliser dans une phase de scoring. Votre code de création de pipeline doit être inclus dans une **fonction Python**.